

Sistema de Gerenciamento de TI - Documentação Base Completa

COPPEAD/UFRJ - Versão 1.0

Data: 26 de janeiro de 2026

Versão: 1.0 - Documento Base do Sistema

Status: Pronto para Implementação

▮ Sumário Executivo

Este documento apresenta a especificação técnica completa do **Sistema de Gerenciamento de TI (ITSM)** desenvolvido para o COPPEAD/UFRJ. Trata-se de uma solução moderna, escalável e modular baseada em princípios ITIL, arquitetura de microserviços e boas práticas de engenharia de software.

O sistema foi projetado para modernizar a gestão de suporte técnico, inventário de ativos, operação de serviços e entrega de valor da TI para usuários internos. A abordagem modular permite implementação incremental, começando pelos módulos core e expandindo conforme demanda operacional.

▮ 1. Visão Geral e Propósito

1.1 Objetivo Principal

O sistema centraliza e automatiza processos de TI, substituindo a abordagem fragmentada atual por uma plataforma integrada que segue a metodologia ITIL. Proporciona:

- **Visibilidade total** do portfólio de ativos e serviços
- **Rastreabilidade completa** de incidentes, requisições e mudanças
- **Automação inteligente** de fluxos repetitivos
- **Inteligência de negócio** via dashboards e KPIs
- **Experiência do usuário** simplificada e intuitiva

1.2 Princípios Norteadores

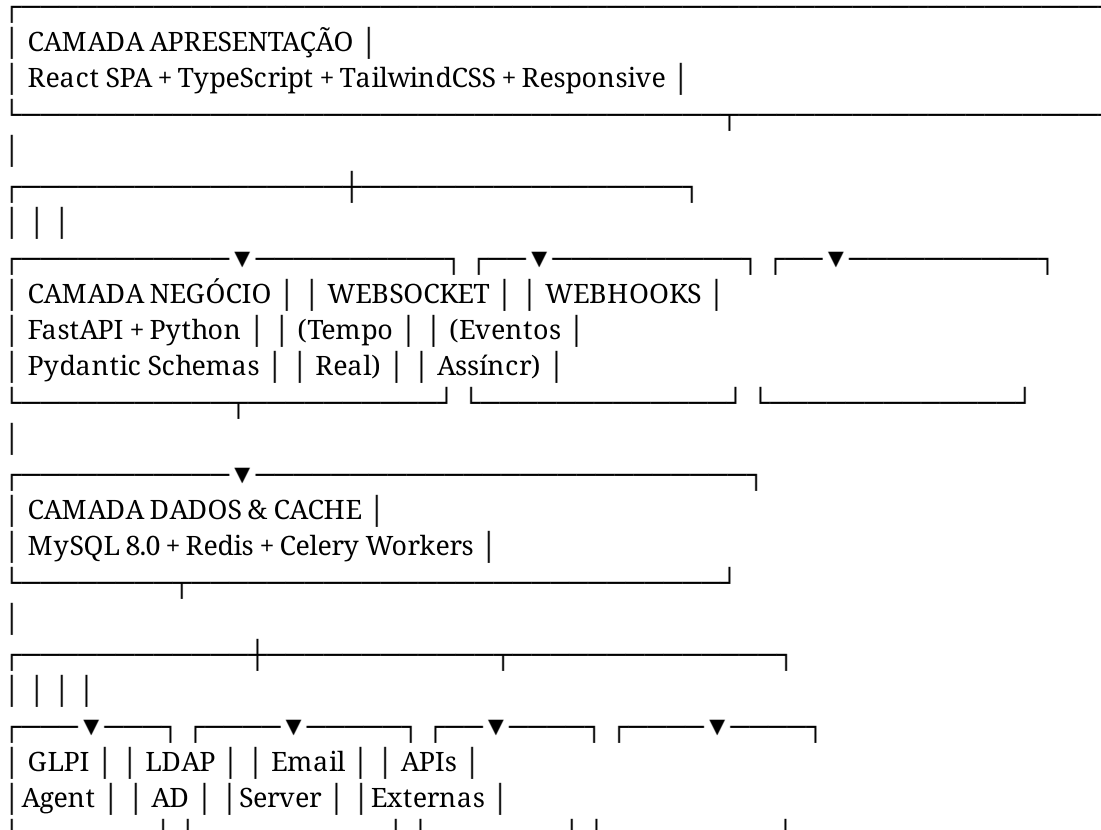
1. **ITIL Adaptado:** Práticas ITIL 4 ajustadas ao contexto acadêmico
2. **Modularidade:** Componentes independentes e acoplamento fraco
3. **Escalabilidade:** Arquitetura pronta para crescimento
4. **Segurança:** Autenticação, autorização e auditoria robustas
5. **Automação:** Redução de tarefas manuais repetitivas
6. **Experiência do Usuário:** Interface intuitiva e responsiva
7. **Manutenibilidade:** Código limpo, documentado e testado

1.3 Público-Alvo

- **Usuários Finais:** Alunos, professores e colaboradores do COPPEAD/UFRJ
- **Técnicos de Suporte:** Equipe de TI que atende tickets
- **Gestores de TI:** Supervisores e coordenadores que monitoram KPIs
- **Administradores do Sistema:** Equipe responsável por manutenção e configurações

2. Arquitetura do Sistema

2.1 Visão Geral da Arquitetura



2.2 Tecnologias Utilizadas

Frontend

Tecnologia	Versão	Propósito
React	18+	Framework SPA (Single Page Application)
TypeScript	5+	Tipagem estática e segurança de tipo
TailwindCSS	3+	Estilização utilitária e responsividade
Axios	1+	Cliente HTTP para consumo de API
React Query	5+	Cache e sincronização de dados
React Router	6+	Roteamento de páginas
Zustand ou Redux	Latest	Gerenciamento de estado global
Socket.io	4+	Comunicação em tempo real (WebSocket)

Arquitetura Frontend:

- Componentes reutilizáveis em padrão Atomic Design
- Separação clara entre smart (conectadas) e dumb (presentation) components
- Temas claro/escuro com CSS Variables
- Resposta total em mobile, tablet e desktop
- Acessibilidade WCAG 2.1 AA

Backend

Tecnologia	Versão	Propósito
Python	3.11+	Linguagem base
FastAPI	0.104+	Framework web assíncrono e high-performance
SQLAlchemy	2.0+	ORM para abstração de banco de dados
Pydantic	2.0+	Validação e serialização de dados
Celery	5.3+	Fila de tarefas distribuída
Passlib + bcrypt	Latest	Hashing seguro de senhas
PyJWT	Latest	Geração e validação de JWT
LDAP3	Latest	Integração com LDAP/Active Directory
python-multipart	Latest	Upload de arquivos
Email-validator	Latest	Validação de e-mail

Arquitetura Backend:

- Estrutura em camadas (routes → services → models)
- Dependency Injection para testabilidade
- Validação de entrada em Pydantic models
- Tratamento de erro centralizado e consistente
- Rate limiting e CORS configurável
- Logging estruturado (JSON)

Banco de Dados

Componente	Tecnologia	Propósito
Dados Primários	MySQL 8.0+	SGBD relacional principal
Cache	Redis 7.0+	Cache distribuído e sessões JWT
Fila	Redis	Suporte a Celery (tarefas assíncronas)
Backup	mysqldump + shell script	Backup automático diário

Infraestrutura

Componente	Tecnologia	Propósito
Orquestração	Docker Compose	Containerização e orquestração local
Servidor Web	Nginx	Reverse proxy e servidor estático
OS	Ubuntu Server 24.04 LTS	Sistema operacional
CI/CD	GitHub Actions	Automação de build, teste e deploy
Versionamento	Git + GitHub	Controle de versão

2.3 Integrações Externas

GLPI (Inventory Management)

- **Propósito:** Sincronização bidirecional de ativos e inventário
- **Protocolo:** GLPI API REST
- **Frequência:** Sincronização noturna (Celery job) + on-demand
- **Dados sincronizados:** Computadores, periféricos, software, rede
- **Plugin:** FusionInventory para coleta automatizada

LDAP/Active Directory

- **Propósito:** Autenticação centralizada e sincronização de usuários
- **Protocolo:** LDAP Protocol (porta 389/636)
- **Frequência:** Sincronização diária (02:00 UTC)
- **Dados sincronizados:** Nome completo, e-mail, grupo, matrícula
- **Fallback:** Admin local em caso de falha LDAP

Email

- **Propósito:** Notificações (tickets, aprovações, SLA crítico)
- **Servidor:** Suportado via credenciais SMTP
- **Templates:** Renderização em Jinja2 com CSS inlined
- **Fila:** Celery para envio assíncrono

APIs Externas (Futuro)

- **Slack/Teams:** Notificações em tempo real
- **GitHub:** Webhook para vincular commits a tickets
- **Monitoramento:** Zabbix/Prometheus para alertas

3. Módulos Funcionais Detalhados

3.1 Módulo de Autenticação e Autorização

3.1.1 Descrição Geral

Gerencia identidade dos usuários, sessões seguras e controle de acesso granular baseado em papéis (RBAC - Role-Based Access Control).

3.1.2 Componentes Principais

Login via LDAP (Autenticação)

Fluxo:

1. Usuário insere credenciais (usuário/senha)
 2. Sistema conecta a servidor LDAP
 3. Valida credenciais contra AD/LDAP
 4. Se válido: cria sessão JWT e retorna token
 5. Se inválido: registra tentativa falhada
 6. Se LDAP indisponível: fallback para admin local
- **Segurança:** Conexão SSL/TLS para LDAP
 - **Validação:** Regex para username, senha com mínimo 8 caracteres
 - **Rate Limiting:** Máximo 5 tentativas falhadas por IP em 15 minutos

Sessões JWT Revogáveis

- **Token Structure:** Header (alg, typ) + Payload (sub, email, roles, iat, exp) + Signature
- **TTL:** 2 horas (access token), 30 dias (refresh token)
- **Armazenamento:** Refresh token em Redis com revoke list
- **Refresh:** Automático quando access token expira
- **Revoke:** Logout imediato adicionando token em blacklist Redis

RBAC - Role-Based Access Control

Entidades:

- User: id, ldap_username, email, full_name, is_active
- Role: id, name (admin, técnico, usuário, gerente)
- Permission: id, name, resource, action (create, read, update, delete)
- RolePermission: role_id, permission_id
- UserRole: user_id, role_id
- **Papéis Padrão:**
 - Admin: Acesso total
 - Técnico: Atribuição, resolução de tickets, visualização de assets
 - Usuário: Abertura de tickets, visualização de próprios tickets
 - Gerente: Visualização de dashboards, relatórios, aprovações
 - Aprovador: Aprovação de requisições e mudanças
- **Verificação:** Decorador Python `@require_permission("recurso", "ação")`

Integração de Grupos LDAP

- Sincronização diária de grupos LDAP para roles do sistema
- Exemplo: CN=TI-Técnicos,OU=... → role "técnico"
- Usuários herdados automaticamente do grupo LDAP

3.1.3 Tecnologias Específicas

- **LDAP3:** Cliente Python para conexão LDAP/AD
- **PyJWT:** Geração e validação de tokens JWT
- **Passlib + bcrypt:** Hash de senhas para admin local (apenas fallback)
- **FastAPI Depends:** Injeção de dependência para autenticação

3.1.4 Endpoints e Operações

Operação	Método	Endpoint	Descrição
Login	POST	/api/auth/login	Autentica e retorna JWT
Refresh Token	POST	/api/auth/refresh	Renova access token
Logout	POST	/api/auth/logout	Revoga sessão
Perfil Atual	GET	/api/auth/me	Retorna dados do usuário
Trocar Senha	POST	/api/auth/change-password	Altera senha (LDAP ou local)

3.2 Módulo de Gerenciamento de Usuários

3.2.1 Descrição Geral

Centraliza gestão de usuários, sincronização com LDAP, controle de ativação/desativação e perfis de acesso.

3.2.2 Componentes Principais

Sincronização LDAP Automatizada

Celery task executada diariamente às 02:00 UTC

```
@periodic_task(run_every=crontab(hour=2, minute=0))
def sync_users_from_ldap():
    """
```

1. Conecta a servidor LDAP
 2. Lista todos os usuários em CN=Users
 3. Para cada usuário:
 - Cria ou atualiza em banco local
 - Sincroniza grupos LDAP → roles
 - Marca como is_active=True
 4. Usuários não encontrados em LDAP: marca is_active=False
 5. Registra resultado em audit_log
- ```
 """
```

- **Frequência:** Diária, fora do horário de pico
- **Timeout:** 5 minutos com retry exponencial
- **Logging:** Cada mudança registrada com before/after

#### Atributos de Usuário

```
CREATE TABLE users (
id UUID PRIMARY KEY,
ldap_username VARCHAR(255) UNIQUE,
email VARCHAR(255) UNIQUE,
full_name VARCHAR(255),
matricula VARCHAR(50),
phone VARCHAR(20),
department VARCHAR(255),
manager_id UUID (FK users.id),
is_active BOOLEAN DEFAULT True,
created_at TIMESTAMP,
updated_at TIMESTAMP,
last_login TIMESTAMP,
sync_status ENUM('SYNCED', 'PENDING', 'ERROR'),
sync_error_message TEXT
);
```

#### Desativação Automática



- Usuários ausentes em LDAP por mais de 30 dias: desativados automaticamente
- Desativação não deleta dados históricos (soft delete via is\_active)
- Reativação automática se usuário reaparece em LDAP

### Perfil de Usuário (Self-Service)

GET /api/users/me

PUT /api/users/me (atualizar telefone, foto)

POST /api/users/me/avatar (upload de avatar)

- Visualização de tickets pessoais
- Histórico de solicitações
- Preferências de notificação

#### 3.2.3 Tecnologias Específicas

- **LDAP3:** Protocolo LDAP
- **APScheduler ou Celery Beat:** Agendamento de sincronização
- **Pydantic:** Validação de dados de usuário

#### 3.2.4 Endpoints e Operações

| Operação                | Método | Endpoint                          | Descrição     |
|-------------------------|--------|-----------------------------------|---------------|
| Listar Usuários         | GET    | /api/admin/users                  | Apenas admin  |
| Buscar Usuário          | GET    | /api/admin/users/{user_id}        | Detalhes      |
| Atualizar Usuário       | PUT    | /api/admin/users/{user_id}        | Admin ou self |
| Ativar/Desativar        | PATCH  | /api/admin/users/{user_id}/status | Admin apenas  |
| Forçar Sincronização    | POST   | /api/admin/users/sync-ldap        | Admin apenas  |
| Status de Sincronização | GET    | /api/admin/users/sync-status      | Admin apenas  |

### 3.3 Módulo de Inventário de Ativos

### 3.3.1 Descrição Geral

Mantém registro centralizado de todos os ativos de TI, integrando com GLPI e fornecendo visibilidade completa do portfólio.

### 3.3.2 Componentes Principais

#### Tipos de Ativos Suportados

| Tipo              | Descrição              | Exemplos                        |
|-------------------|------------------------|---------------------------------|
| <b>Hardware</b>   | Equipamentos físicos   | Computadores, servidores        |
| <b>Periférico</b> | Acessórios conectados  | Monitor, teclado, mouse, webcam |
| <b>Rede</b>       | Equipamentos de rede   | Switch, roteador, WAP           |
| <b>Móvel</b>      | Dispositivos móveis    | Notebook, tablet, smartphone    |
| <b>Software</b>   | Aplicações instaladas  | Windows, Office, navegadores    |
| <b>Servidor</b>   | Infraestrutura crítica | VMs, servidores físicos         |
| <b>Consumível</b> | Itens descartáveis     | Toner, papel, cabos             |

#### Estados do Ativo

```
ENUM asset_status {
'DISPONÍVEL', -- Ativo e pronto para alocação
'EM_USO', -- Alocado a um usuário
'MANUTENÇÃO', -- Enviado para reparo
'APOSENTADO', -- Fora de uso/descartado
'RESERVADO' -- Separado para projeto
}
```

#### Estrutura de Dados do Ativo

```
CREATE TABLE assets (
 id UUID PRIMARY KEY,
 asset_type ENUM (veja tabela acima),
 name VARCHAR(255) NOT NULL,
 description TEXT,
 serial_number VARCHAR(255) UNIQUE,
 manufacturer VARCHAR(255),
 model VARCHAR(255),
 acquisition_date DATE,
 warranty_until DATE,
```

```
cost DECIMAL(12,2),
location_id UUID (FK locations.id),
assigned_to_user_id UUID (FK users.id),
status asset_status DEFAULT 'DISPONÍVEL',
```

```
-- Atributos de Rede
ip_address INET,
mac_address VARCHAR(17),
hostname VARCHAR(255),

-- Atributos de Virtualizacao
host_id UUID (FK assets.id), -- VM hospedada em...

-- Auditoria
created_by_user_id UUID,
created_at TIMESTAMP,
updated_at TIMESTAMP,
deleted_at TIMESTAMP -- Soft delete
```

```
);
```

### Sincronização com GLPI

Fluxo:

1. Noturno: Job Celery conecta a GLPI via API
2. Requisita lista de ativos (computadores, periféricos, etc.)
3. Compara com banco local:
  - Novos em GLPI: insere em local
  - Modificados: atualiza
  - Deletados em GLPI: marca deleted\_at em local
4. Registra resultado da sincronização
5. Envia alerta se erros detectados

### Ciclo de Vida do Ativo

Aquisição

↓

Disponível → Em Uso → Manutenção → Em Uso → Aposentado

↑ ↓

└────────────────── (substituição) ─────────────────┘

- **Alertas automáticos:** Garantia expirando, idade > 5 anos
- **Relacionamento com Tickets:** Asset vinculado ao ticket que relata problema

### 3.3.3 Tecnologias Específicas

- **SQLAlchemy Relationships:** Relacionamentos entre assets
- **GLPI REST API:** Sincronização bidirecional
- **Celery Beat:** Job agendado de sincronização
- **PostgreSQL INET:** Validação de endereço IP

### 3.3.4 Endpoints e Operações

| Operação         | Método | Endpoint                        | Descrição                   |
|------------------|--------|---------------------------------|-----------------------------|
| Listar Ativos    | GET    | /api/assets                     | Com filtros                 |
| Detalhe do Ativo | GET    | /api/assets/{asset_id}          | Histórico + relacionamentos |
| Criar Ativo      | POST   | /api/assets                     | Técnico/Admin               |
| Atualizar Ativo  | PUT    | /api/assets/{asset_id}          | Modificar dados             |
| Mudar Status     | PATCH  | /api/assets/{asset_id}/status   | Disponível/Manutenção/etc   |
| Alocar a Usuário | POST   | /api/assets/{asset_id}/assign   | Técnico/Admin               |
| Desalocar        | POST   | /api/assets/{asset_id}/unassign | Técnico/Admin               |
| Sincronizar GLPI | POST   | /api/admin/assets/sync-glpi     | Admin apenas                |

---

## 3.4 Módulo CMDB e Relacionamentos

### 3.4.1 Descrição Geral

CMDB (Configuration Management Database) mantém Item de Configuração (CI) com relacionamentos complexos, independente de ativos físicos. Essencial para análise de impacto e resolução de incidentes.

### 3.4.2 Componentes Principais

#### CI (Configuration Item)

Um CI é um elemento da infraestrutura que pode ser:

- Ativo (hardware, software)
- Serviço (email, VPN, Wi-Fi)
- Componente lógico (banco de dados, aplicação)

- Documentação (política, SOP)

```
CREATE TABLE configuration_items (
 id UUID PRIMARY KEY,
 name VARCHAR(255) NOT NULL,
 description TEXT,
 ci_type ENUM (
 'HARDWARE', 'SOFTWARE', 'SERVIÇO',
 'APLICAÇÃO', 'BD', 'REDE', 'DOCUMENTAÇÃO'
),
 status ENUM ('ATIVO', 'INATIVO', 'DEPRECIADO'),
 owner_id UUID (FK users.id),
 criticality ENUM (1, 2, 3, 4, 5) -- 5 = crítico,
 impact_level INT DEFAULT 0, -- Tickets afetados se down
```

```
-- Rastreabilidade
asset_id UUID (FK assets.id, nullable),
created_at TIMESTAMP,
updated_at TIMESTAMP
```

```
);
```

### Relacionamentos CI

```
CREATE TABLE ci_relationships (
 id UUID PRIMARY KEY,
 source_ci_id UUID NOT NULL (FK configuration_items.id),
 target_ci_id UUID NOT NULL (FK configuration_items.id),
 relationship_type ENUM (
 'HOSPEDA', -- VM hospedada em Host
 'DEPENDE_DE', -- Aplicação depende de BD
 'FORNECE_ACESSO_A', -- LDAP fornece acesso a...
 'USA', -- Serviço usa componente
 'CONECTA_A', -- Rede conecta a...
 'FALHA_AFETA' -- Falha afeta outro CI
),
 impact_propagation BOOLEAN DEFAULT False,
 created_at TIMESTAMP,
 updated_at TIMESTAMP,
```

```
 UNIQUE (source_ci_id, target_ci_id, relationship_type)
```

```
);
```

### Exemplos de Relacionamentos

Hardware: Servidor UFRJ-TST-001  
 └─ Hospeda → CI: Linux Ubuntu 22.04

- |— Hospeda → CI: MySQL 8.0
- |— Hospeda → CI: Aplicação GLPI

Aplicação: GLPI (serviço de helpdesk)

- |— Depende de → CI: MySQL 8.0
- |— Depende de → CI: Redis 7.0
- |— Fornece acesso a → CI: LDAP
- |— Usa → CI: Storage NFS

### **Propagação de Impacto**

Quando um CI crítico é afetado (ex: BD offline), o sistema:

1. Identifica todos os CIs que dependem dele
2. Marca-os como "AFETADO"
3. Escalona tickets relacionados
4. Notifica gestores de TI
5. Registra timeline do incidente

#### **3.4.3 Visualização em Árvore**

- Interface web mostra CIs em estrutura hierárquica
- Visualização de relacionamentos com ícones coloridos
- Zoom em CI específico para ver dependências (2 níveis up/down)
- Busca de impacto: "Se isso falhar, o que quebra?"

#### **3.4.4 Tecnologias Específicas**

- **SQLAlchemy Graph**: Relacionamentos de grafo
- **D3.js ou Cytoscape.js**: Visualização gráfica (future)
- **Algoritmo BFS**: Cálculo de impacto em cascata

#### **3.4.5 Endpoints e Operações**

| <b>Operação</b>        | <b>Método</b> | <b>Endpoint</b>                     | <b>Descrição</b>        |
|------------------------|---------------|-------------------------------------|-------------------------|
| Listar CIs             | GET           | /api/cmdb/cis                       | Com filtros             |
| Detalhe CI             | GET           | /api/cmdb/cis/{ci_id}               | Incluir relacionamentos |
| Criar CI               | POST          | /api/cmdb/cis                       | Técnico/Admin           |
| Atualizar CI           | PUT           | /api/cmdb/cis/{ci_id}               | Modificar               |
| Listar Relacionamentos | GET           | /api/cmdb/cis/{ci_id}/relationships | Dependências            |
| Criar Relacionamento   | POST          | /api/cmdb/relationships             | Vincular CIs            |
| Remover Relacionamento | DELETE        | /api/cmdb/relationships/{rel_id}    | Desvincula              |
| Análise de Impacto     | GET           | /api/cmdb/cis/{ci_id}/impact        | Mostra cascata          |

### 3.5 Sistema de Chamados (ITSM)

#### 3.5.1 Descrição Geral

Coração do sistema. Gerencia incidentes, requisições, problemas e mudanças seguindo metodologia ITIL.

#### 3.5.2 Tipos de Chamados

##### 3.5.2.1 Incidente

Definição: Interrupção não planejada de serviço ou redução de qualidade.

Exemplo: "Não consigo fazer login no Gmail corporativo"

Fluxo:

1. Abertura: Usuário ou técnico abre ticket
2. Triagem: Categorização + priorização (urgência × impacto)
3. Atribuição: Automática baseada em carga + expertise
4. Investigação: Técnico adiciona comentários internos/públicos
5. Resolução: Técnico resolve problema
6. Fechamento: Usuário confirma resolução e opcionalmente fornece feedback

Estrutura de Dados:

```

CREATE TABLE tickets (
id UUID PRIMARY KEY,
ticket_number INT AUTO_INCREMENT UNIQUE, -- #12345
type ENUM ('INCIDENTE', 'REQUISIÇÃO', 'PROBLEMA', 'MUDANÇA'),
status ENUM (
'NOVO', -- Acabou de ser aberto
'TRIAGEM', -- Aguardando classificação
'EM_ANDAMENTO', -- Técnico está trabalhando
'AGUARDANDO', -- Aguardando info do usuário/recurso
'RESOLVIDO', -- Técnico resolveu, aguarda confirmação
'FECHADO', -- Confirmado resolvido
'CANCELADO' -- Cancelado por usuário/inatividade
),

```

```

-- Identificação
title VARCHAR(255) NOT NULL,
description TEXT NOT NULL,
priority ENUM ('P1_CRÍTICO', 'P2_ALTO', 'P3_MÉDIO', 'P4_BAIXO'),

```

```

-- Prioridade Cálculo
urgency ENUM ('CRÍTICA', 'ALTA', 'MÉDIA', 'BAIXA'),
impact ENUM ('MÚLTIPLOS', 'ALGUNS', 'INDIVIDUAL'),
-- priority = matrix[urgency][impact]

```

```

-- Rastreabilidade
opened_by_user_id UUID NOT NULL (FK users.id),
assigned_to_user_id UUID (FK users.id), -- Técnico
assigned_to_group_id UUID (FK groups.id),
category_id UUID (FK categories.id),
asset_id UUID (FK assets.id), -- Ativo relacionado
ci_id UUID (FK configuration_items.id), -- CI relacionado

```

```

-- SLA
sla_id UUID (FK slas.id),
sla_target_time TIMESTAMP,
sla_status ENUM ('OK', 'CRÍTICO', 'EXPIRADO'),

```

```

-- Resolução
solution TEXT,
root_cause TEXT, -- Para problemas
resolved_at TIMESTAMP,

```



```
closed_at TIMESTAMP,

-- Auditoria
created_at TIMESTAMP,
updated_at TIMESTAMP,
deleted_at TIMESTAMP -- Soft delete
```

);

### 3.5.2.2 Requisição

Definição: Solicitação formal de um serviço (ex: criar conta, instalar software).

Exemplo: "Solicito instalação de Visual Studio Code"

Fluxo:

1. Seleção de Catálogo: Usuário escolhe serviço do catálogo
2. Preenchimento: Formulário dinâmico (variável por tipo)
3. Aprovação: Segue workflow de aprovação (1 ou N níveis)
4. Execução: Técnico recebe como ticket + subtasks automáticas
5. Encerramento: Após conclusão, pede confirmação ao usuário

Estrutura:

- Tabelas: service\_requests, service\_request\_approvals
- Herança de alguns campos de ticket
- Campo adicional: requested\_by (quem pediu)

### 3.5.2.3 Problema

Definição: Causa raiz de um ou mais incidentes.

Exemplo: "Falha no driver de impressora gera 15 incidentes"

Fluxo:

1. Criação: Técnico cria problema associado a incidente
2. RCA: Pesquisa de causa raiz + documentação
3. Workaround: Solução temporária para usuários
4. Mudança: Se necessário, cria RFC associada
5. Encerramento: Após resolução completa

Estrutura:

- Campos: root\_cause, workaround, known\_error
- Relacionamento: 1 problema : N incidentes
- Status: NOVO → ANÁLISE → SOLUÇÃO → ENCERRADO

### 3.5.2.4 Mudança (Change)

Definição: Modificação controlada de infraestrutura/serviços.

Exemplo: "Upgrade de MySQL 5.7 para 8.0"

Fluxo:

1. RFC: Criar Request for Change com detalhes
  - Descrição da mudança
  - Razão / Benefícios
  - Impacto estimado
  - Janela de mudança
  - Plano de rollback
2. Avaliação: CAB (Change Advisory Board) revisa
  - Complexidade
  - Riscos
  - Recurso necessário
3. Aprovação: Aprovador executa decisão (APROVADA/REJEITADA)
4. Execução: Técnico implementa no período programado
  - Checklist pré-mudança
  - Registra ações
  - Valida sucesso
5. Revisão: Pós-mudança (24-48h)
  - Confirma estabilidade
  - Documenta lições aprendidas
  - Fecha change request

Estrutura:

- change\_requests: id, title, description, risk\_level, impact
- change\_schedule: change\_id, scheduled\_start, scheduled\_end
- change\_tasks: subtasks da mudança
- Status: NOVO → AGENDADA → APROVADA → EM\_EXECUÇÃO → VALIDADA → FECHADA

### 3.5.3 Priorização

Matriz de Priorização (3×3):

| IMPACTO |  |  |
|---------|--|--|
|---------|--|--|

URGÊNCIA Múltiplos | Alguns | Individual

---

Crítica | P1 | P2 | P3

---

Alta | P2 | P2 | P3

---

Média | P2 | P3 | P4

---

Baixa | P3 | P4 | P4

- **P1 (Crítico):** SLA 4 horas, tela vermelha no dashboard
- **P2 (Alto):** SLA 8 horas, notificação frequente
- **P3 (Médio):** SLA 24 horas, prioridade normal

- **P4 (Baixo):** SLA 48 horas, fila simples

#### 3.5.4 Comentários e Interações

```
CREATE TABLE ticket_comments (
 id UUID PRIMARY KEY,
 ticket_id UUID NOT NULL (FK tickets.id),
 author_id UUID NOT NULL (FK users.id),
 content TEXT NOT NULL,
 is_public BOOLEAN DEFAULT False, -- Visível ao usuário?
 mentions_user_id UUID[] (FK users.id), -- @mention
 attachments JSON, -- Array de file_ids
 created_at TIMESTAMP,
 updated_at TIMESTAMP,
 deleted_at TIMESTAMP
);
```

- **Comentário Público:** Visível ao solicitante e equipe
- **Comentário Interno:** Apenas equipe técnica
- **@Mention:** Notifica usuário mencionado
- **Anexos:** Upload de arquivos (logs, screenshots)
- **Edição:** Preserva histórico de edições

#### 3.5.5 Feedback e Satisfação

```
CREATE TABLE ticket_feedback (
 id UUID PRIMARY KEY,
 ticket_id UUID NOT NULL UNIQUE (FK tickets.id),
 rating INT CHECK (rating BETWEEN 1 AND 5),
 comment TEXT,
 would_recommend BOOLEAN,
 created_at TIMESTAMP
);
```

- Enviado após fechamento do ticket
- Opcional + comentário (ex: "Excelente! Resolveu em tempo")
- Alimenta dashboard de NPS por técnico/categoria
- Pode bloquear fechamento definitivo (feedback obrigatório?)

#### 3.5.6 Tarefas do Ticket (Subtasks)

Para requisições com execução complexa:

```
CREATE TABLE ticket_tasks (
 id UUID PRIMARY KEY,
 ticket_id UUID NOT NULL (FK tickets.id),
 title VARCHAR(255),
 description TEXT,
 assigned_to_user_id UUID (FK users.id),
 status ENUM ('PENDENTE', 'EM_ANDAMENTO', 'CONCLUÍDO'),
 due_date DATE,
 created_at TIMESTAMP
);
```

Exemplo: Requisição "Onboarding novo colaborador"

- Task 1: Criar conta de usuário no AD
- Task 2: Criar mailbox Exchange
- Task 3: Adicionar a grupos de segurança
- Task 4: Entregar notebook + periféricos
- Task 5: Treinamento de sistemas

### 3.5.7 Tecnologias Específicas

- **SQLAlchemy Enums:** Validação de valores
- **Pydantic Discriminated Union:** Suporta tipos variados de tickets
- **APScheduler:** Auto-fechamento de tickets inativos
- **Elasticsearch** (futuro): Busca full-text em tickets históricos

### 3.5.8 Endpoints e Operações

| Operação             | Método | Endpoint                          | Descrição           |
|----------------------|--------|-----------------------------------|---------------------|
| Listar Tickets       | GET    | /api/tickets                      | Com filtros         |
| Criar Ticket         | POST   | /api/tickets                      | Usuário/técnico     |
| Detalhe Ticket       | GET    | /api/tickets/{ticket_id}          | Incluir comentários |
| Atualizar Ticket     | PUT    | /api/tickets/{ticket_id}          | Modificar           |
| Mudar Status         | PATCH  | /api/tickets/{ticket_id}/status   | Transição estado    |
| Adicionar Comentário | POST   | /api/tickets/{ticket_id}/comments | Comentário          |
| Fornecer Feedback    | POST   | /api/tickets/{ticket_id}/feedback | Avaliação           |
| Atribuir Técnico     | POST   | /api/tickets/{ticket_id}/assign   | Atribuição          |
| Escalar              | POST   | /api/tickets/{ticket_id}/escalate | Escalonamento       |

## 3.6 Catálogo de Serviços

### 3.6.1 Descrição Geral

Interface self-service onde usuários solicitam serviços de TI (criar conta, instalar software, manutenção, etc.). Cada item define formulário, fluxo de aprovação, SLA e tarefas.

### 3.6.2 Componentes Principais

#### Estrutura de Catálogo

```
CREATE TABLE service_catalogs (
 id UUID PRIMARY KEY,
 name VARCHAR(255) NOT NULL,
 description TEXT,
 icon VARCHAR(255), -- URL ou emoji
 order INT,
 is_active BOOLEAN DEFAULT True
);
```

```
CREATE TABLE service_items (
 id UUID PRIMARY KEY,
 catalog_id UUID NOT NULL (FK service_catalogs.id),
 name VARCHAR(255) NOT NULL,
 description TEXT,
 icon VARCHAR(255),
 order INT,
```

```
-- Acesso
visible_to_groups UUID[], -- Quem vê this item

-- Formulário
form_schema JSON, -- Pydantic schema + validações

-- Aprovação
requires_approval BOOLEAN DEFAULT False,
approval_group_id UUID (FK groups.id), -- Quem aprova?
approval_levels INT DEFAULT 1, -- 1 ou N aprovadores

-- SLA
sla_id UUID (FK slas.id), -- SLA específico deste serviço

-- Execução
auto_tasks_template JSON, -- Tasks geradas ao criar requisição

-- Auditoria
```

```
created_at TIMESTAMP,
updated_at TIMESTAMP
```

```
);
```

### Exemplo: Item "Instalar Software"

```
{
 "name": "Instalar Software",
 "description": "Solicite instalação de software no seu computador",
 "form_schema": {
 "type": "object",
 "properties": {
 "software_name": {
 "type": "string",
 "title": "Nome do Software",
 "enum": ["Visual Studio Code", "Python 3.11", "Docker"]
 },
 "reason": {
 "type": "string",
 "title": "Motivo/Justificativa"
 },
 "target_computer": {
 "type": "string",
 "title": "Computador (selecione seu PC)"
 // Busca assets do usuário
 }
 },
 "required": ["software_name", "reason", "target_computer"]
 },
 "requires_approval": true,
 "approval_group_id": "grp_ti_software",
 "auto_tasks": [
 {
 "title": "Validar licença",
 "assigned_to": "grp_ti_licensing"
 },
 {
 "title": "Instalar em máquina",
 "assigned_to": "grp_ti_deskside"
 }
]
}
```

### Fluxo Completo de Requisição

#### 1. SELEÇÃO

Usuário clica "Instalar Software" no catálogo

#### 2. PREENCHIMENTO

Sistema exibe formulário dinâmico baseado em form\_schema

- Validação real-time (campo obrigatório, type check)
  - Dependências entre campos (se escolhe Windows... mostrar versão)
3. RESUMO E CONFIRMAÇÃO  
Revisar dados antes de enviar  
"Solicitar instalação de VS Code em UFRJ-PC-12345"
  4. ABERTURA DE REQUISIÇÃO  
Sistema cria ticket tipo REQUISIÇÃO com status NOVO  
Tarefas automáticas são geradas
  5. APROVAÇÃO (se configurado)  
approval\_request criada para grupo aprovador  
Aprovador recebe notificação  
Pode aprovar/rejeitar com comentário
  6. APROVADO → EXECUÇÃO  
Ticket passa a EM\_ANDAMENTO  
Tarefas pré-criadas são atribuídas a técnicos  
Ex: "Validar licença" → TI-Software  
"Instalar" → TI-Deskside
  7. EXECUÇÃO  
Técnicos completam tarefas, adicionam comentários  
Usuário pode acompanhar progresso
  8. FECHAMENTO  
Técnico marca RESOLVIDO  
Sistema envia notificação ao usuário  
Pede feedback
  9. FEEDBACK  
Usuário avalia (1-5 estrelas + comentário)  
Ticket encerrado definitivamente

### 3.6.3 Visibilidade e Controle de Acesso

## Lógica de visibilidade

GET /api/service-catalog

**Retorna apenas items onde:**

**- is\_active = True**

**- current\_user.groups intersecção  
visible\_to\_groups > 0**

## - OR visible\_to\_groups is empty (público)

- **Catálogo Público:** Visível para todos (ex: "Solicitar Acesso Wi-Fi")
- **Catálogo Restrito:** Apenas grupos específicos (ex: "Requisições Administrativas")
- **Catálogo Gerencial:** Apenas gestores (ex: "Criar Nova Conta Contratado")

### 3.6.4 Templates Automáticos

Ao criar requisição, sistema gera tarefas:

## Exemplo: "Onboarding de novo colaborador"

```
auto_tasks = [
Task(title="Criar conta AD", assigned_to="ti_systems"),
Task(title="Criar mailbox", assigned_to="ti_messaging"),
Task(title="Adicionar grupos segurança", assigned_to="ti_security"),
Task(title="Provisionar notebook", assigned_to="ti_deskside"),
Task(title="Treinar sistemas", assigned_to="ti_training"),
]
```

## Todas criadas com status PENDENTE

## Técnicos recebem notificação

### 3.6.5 Tecnologias Específicas

- **JSON Schema Draft 7:** Validação de formulário dinâmico
- **JSON Editor + React JSON Schema Form:** Frontend
- **Pydantic Discriminated Union:** Validação backend variável

### 3.6.6 Endpoints e Operações



| Operação           | Método | Endpoint                       | Descrição                   |
|--------------------|--------|--------------------------------|-----------------------------|
| Listar Catálogos   | GET    | /api/service-catalog           | Respeitando acesso          |
| Detalhe Item       | GET    | /api/service-items/{item_id}   | Form schema                 |
| Criar Requisição   | POST   | /api/service-requests          | Baseado em item             |
| Listar Requisições | GET    | /api/service-requests          | Do usuário ou todas (admin) |
| Detalhe Requisição | GET    | /api/service-requests/{req_id} | Status + tarefas            |

### 3.7 Base de Conhecimento (KB)

#### 3.7.1 Descrição Geral

Repositório centralizado de artigos, FAQs e soluções. Escalona autoatendimento e reduz carga de suporte.

#### 3.7.2 Componentes Principais

##### Estrutura de Artigos

```
CREATE TABLE knowledge_articles (
 id UUID PRIMARY KEY,
 title VARCHAR(255) NOT NULL,
 slug VARCHAR(255) UNIQUE,
 content TEXT NOT NULL, -- Markdown
 summary TEXT,
 category_id UUID NOT NULL (FK knowledge_categories.id),
 tags VARCHAR(255)[],
```

```
-- Controle
is_published BOOLEAN DEFAULT False,
visibility ENUM ('PÚBLICO', 'INTERNO', 'RESTRITO'),
visible_to_groups UUID[], -- Se RESTRITO

-- Autoria
author_id UUID NOT NULL (FK users.id),
```

```
-- Relevância
view_count INT DEFAULT 0,
helpful_count INT DEFAULT 0,
not_helpful_count INT DEFAULT 0,
```

```
-- Auditoria
created_at TIMESTAMP,
published_at TIMESTAMP,
updated_at TIMESTAMP
```

```
);
```

```
CREATE TABLE knowledge_categories (
id UUID PRIMARY KEY,
name VARCHAR(255),
slug VARCHAR(255),
description TEXT,
parent_id UUID (FK knowledge_categories.id), -- Sub-categorias
order INT
);
```

### Exemplo de Artigo

# Como Resetar Senha do Windows

## Categoria

Sistemas Operacionais > Windows > Autenticação

## Tags

#senha #windows #acesso #reset

## Resumo

Procedimento para resetar senha de login do Windows 10/11.

## Conteúdo (Markdown)

### Pré-requisitos

- Acesso físico ao computador
- Outra conta com privilégios de admin (opcional)

## Passo 1: Acessar tela de login

1. Clique em Windows + L
2. Na tela de login, clique em "Esqueci a senha"

## Passo 2: Resetar via conta Microsoft

Se sua conta for Microsoft:

1. Siga o link "Redefinir senha"
2. Você será redirecionado para [account.microsoft.com](https://account.microsoft.com)
3. ...

## Passo 3: Ou resetar offline (Pro/Enterprise)

Se usar conta local:

1. Use comando `net user username newpassword`
2. ...

## Vinculações

- Problema relacionado: #prob\_123 (Usuários não conseguem fazer login)
- Incidentes associados: 5 últimos

## Versão

v1.2 | Última atualização: 26 jan 2026

**Sugestão Automática ao Abrir Incidente**

# Algoritmo de busca similar (TF-IDF + fuzzy matching)

```
def suggest_articles(ticket_title, ticket_description):
```

```
 """
```

1. Tokeniza e limpa title + description
2. Calcula TF-IDF vs todos os artigos publicados
3. Retorna top 3 artigos mais relevantes
4. Exibe ao usuário: "Esses artigos podem ajudar..."

```
 """
```

```
 return articles_ranked_by_relevance[:3]
```

## Exemplo:

**Ticket: "Não consigo fazer login no Windows"**

**Sugestões:**

- 1. "Como Resetar Senha do Windows" (match: 92%)**
- 2. "Problemas Comuns de Login" (match: 87%)**
- 3. "FAQ - Windows 11" (match: 75%)**

**Feedback e Utilidade**

**Usuário pode marcar artigo como útil/não útil**

```
POST /api/knowledge/{article_id}/feedback
{
 "helpful": true, # ou false
 "comment": "Resolveu meu problema!"
}
```

**Backend atualiza helpful\_count, not\_helpful\_count**

**Algoritmo melhora relevância de artigos mais úteis**

**Acesso Controlado**

- **Público:** Visível a todos (FAQ de autosserviço)
- **Interno:** Apenas técnicos de TI
- **Restrito:** Apenas grupos específicos (ex: artigos de procedimento administrativo)

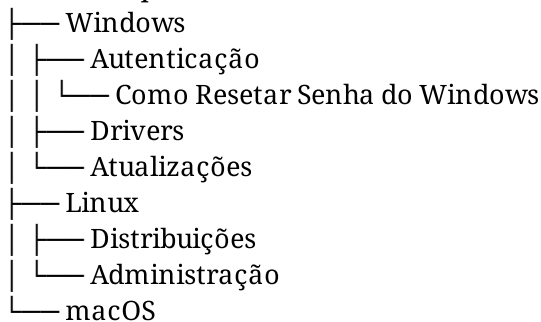
### 3.7.3 Versionamento de Artigos

```
CREATE TABLE knowledge_article_versions (
 id UUID PRIMARY KEY,
 article_id UUID NOT NULL (FK knowledge_articles.id),
 version INT,
 content TEXT,
 changed_by_user_id UUID (FK users.id),
 change_note VARCHAR(255),
 created_at TIMESTAMP
);
```

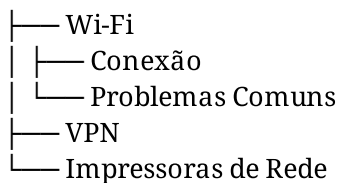
- Cada edição cria versão
- Possibilidade de reverter para versão anterior
- Histórico de mudanças visível

### 3.7.4 Categorização com Hierarquia

Sistemas Operacionais



Redes



### 3.7.5 Tags e Busca

- Tags aplicadas a múltiplos artigos
- Busca por tag, categoria ou full-text
- Autocomplete de tags
- Nuvem de tags mostrando populares

### 3.7.6 Tecnologias Específicas

- **Markdown:** Renderização com highlight de código
- **Elasticsearch** (futuro): Busca full-text com TF-IDF
- **Slugify:** Conversão de título para URL amigável
- **Git-like versioning:** Rastreamento de mudanças

### 3.7.7 Endpoints e Operações

| Operação          | Método | Endpoint                                     | Descrição          |
|-------------------|--------|----------------------------------------------|--------------------|
| Listar Categorias | GET    | /api/knowledge/categories                    | Hierarquia         |
| Listar Artigos    | GET    | /api/knowledge/articles                      | Com paginação      |
| Buscar Artigos    | GET    | /api/knowledge/articles/search               | Filtro + busca     |
| Detalhe Artigo    | GET    | /api/knowledge/articles/{article_id}         | Content + feedback |
| Criar Artigo      | POST   | /api/knowledge/articles                      | Técnico/Admin      |
| Atualizar Artigo  | PUT    | /api/knowledge/articles/{article_id}         | Edição             |
| Publicar Artigo   | PATCH  | /api/knowledge/articles/{article_id}/publish | Draft → Publicado  |
| Marcar Útil       | POST   | /api/knowledge/articles/{article_id}/helpful | Feedback           |
| Sugerir Artigos   | GET    | /api/knowledge/suggest?title=...&desc=...    | Algoritmo TF-IDF   |

## 3.8 Gerenciamento de SLA e Escalonamento

### 3.8.1 Descrição Geral

Define tempos de resposta e resolução por tipo/prioridade de ticket. Automatiza alertas e escalonamento quando SLA é ameaçado ou expirado.

### 3.8.2 Componentes Principais

#### Tabela de SLA

```
CREATE TABLE slas (
 id UUID PRIMARY KEY,
 name VARCHAR(255),
 description TEXT,
```

```
-- Aplicabilidade
ticket_type ENUM ('INCIDENTE', 'REQUISIÇÃO', 'PROBLEMA', 'MUDANÇA'),
priority_level ENUM ('P1', 'P2', 'P3', 'P4'),

-- Tempos (em horas)
response_time INT, -- Tempo para primeira resposta (RTO)
resolution_time INT, -- Tempo para resolução (TTR)

-- Horário Útil
calendar_id UUID (FK calendars.id), -- Referência a calendário

-- Escalação
escalation_actions TEXT, -- JSON de ações

-- Ativo
is_active BOOLEAN DEFAULT True,
created_at TIMESTAMP,
updated_at TIMESTAMP
```

```
);
```

```
-- Exemplo: P1 Incidente
-- response_time: 2 horas
-- resolution_time: 4 horas
-- Conta apenas horário útil (seg-sex 08:00-18:00)
-- Pausado em feriados
```

### **Cálculo de SLA no Ticket**

## **Ao abrir ticket, sistema calcula:**

```
sla_target_response = ticket.created_at + sla.response_time_hours
sla_target_resolution = ticket.created_at + sla.resolution_time_hours
```

## **Cada 5 minutos (Celery job), verifica status:**

```
def check_sla_status(ticket):
 now = datetime.now()
```

```
Pausa SLA se ticket está AGUARDANDO (esperando info do usuário)
if ticket.status == 'AGUARDANDO':
```

```

return 'PAUSADO'

elapsed = calculate_business_hours(ticket.created_at, now)

if not ticket.response_given:
 percentage = (elapsed / sla.response_time) * 100
 if percentage >= 100:
 return 'EXPIRADO' # RTO expirado!
 elif percentage >= 70:
 return 'CRÍTICO' # Alerta

if ticket.status != 'FECHADO':
 percentage = (elapsed / sla.resolution_time) * 100
 if percentage >= 100:
 return 'EXPIRADO' # TTR expirado!
 elif percentage >= 70:
 return 'CRÍTICO'

return 'OK'

```

### Calendário de Horário Útil

```

CREATE TABLE calendars (
 id UUID PRIMARY KEY,
 name VARCHAR(255),
 timezone VARCHAR(50),

```

```

-- Horário base
work_hours_start TIME, -- 08:00
work_hours_end TIME, -- 18:00
work_days ENUM[], -- SEG, TER, QUA, QUI, SEX (não SÁB/DOM)

created_at TIMESTAMP

```

```

);

```

```

CREATE TABLE calendar_holidays (
 id UUID PRIMARY KEY,
 calendar_id UUID NOT NULL (FK calendars.id),
 date DATE,
 name VARCHAR(255), -- "Carnaval", "Ano Novo", etc

```



```
created_at TIMESTAMP
);
```

```
-- Exemplo:
-- Calendar "Brasil"
-- work_hours: 08:00-18:00
-- work_days: SEG-SEX
-- holidays: 01/01, 21/04, 01/05, 25/12, ...
```

### **Cálculo de Horas Úteis**

```
def calculate_business_hours(start: datetime, end: datetime, calendar: Calendar) -> float:
 """
```

Calcula horas úteis entre start e end, respeitando:

- Horário de trabalho configurado
- Dias da semana (seg-sex)
- Feriados do calendário

```
 """
```

```
Exemplo:
```

```
start: seg 10:00, end: ter 11:00
```

```
Horas úteis = 8 (seg 10:00-18:00) + 3 (ter 08:00-11:00) = 11h
```

```
NÃO conta finais de semana nem feriados
```

### **Escalonamento Automático**

Quando SLA atinge limiares, ações automáticas são disparadas:

```
{
 "sla_id": "sla_p1",
 "escalation_rules": [
 {
 "trigger": "70% of resolution_time",
 "actions": [
 {
 "type": "alert",
 "target": "assigned_technician",
 "message": "Alerta: SLA crítico para P1 em 70%"
 },
 {
 "type": "escalate_to_group",
 "target_group": "ti_management",
 "message": "Ticket P1 crítico, escalando para gerência"
 },
 {
 "type": "increase_priority",
 "new_priority": "P1_CRÍTICO"
 }
]
 },
 {
 "trigger": "100% of resolution_time (SLA expired)",
 "actions": [
```

```
{
 "type": "alert",
 "severity": "CRITICAL",
 "target": "manager"
},
{
 "type": "reassign",
 "target_group": "ti_senior_team",
 "reason": "SLA expirado, escalando para senior"
},
{
 "type": "create_incident",
 "description": "SLA violado para ticket #{ticket_number}"
}
]
}
```

## Status Visual de SLA

### 3.8.3 Tecnologías Específicas

- **APScheduler ou Celery Beat:** Job recorrente de verificação SLA
- **Python datetime com pytz:** Cálculo de horas úteis com timezone
- **Redis:** Cache de calendários para performance

### 3.8.4 Endpoints e Operações

| Operação             | Método | Endpoint                            | Descrição    |
|----------------------|--------|-------------------------------------|--------------|
| Listar SLAs          | GET    | /api/slas                           | Admin apenas |
| Criar SLA            | POST   | /api/slas                           | Admin        |
| Atualizar SLA        | PUT    | /api/slas/{sla_id}                  | Admin        |
| Listar Calendários   | GET    | /api/calendars                      | Admin        |
| Criar Calendário     | POST   | /api/calendars                      | Admin        |
| Adicionar Feriado    | POST   | /api/calendars/{cal_id}/holidays    | Admin        |
| Verificar SLA Ticket | GET    | /api/tickets/{ticket_id}/sla-status | Real-time    |

---

### 3.9 Relatórios e Dashboards

#### 3.9.1 Descrição Geral

Fornece visibilidade de KPIs e métricas operacionais. Dashboards são personalizáveis, com filtros avançados e exportação.

#### 3.9.2 Componentes Principais

##### KPIs Principais



|                                                                                                                           |  |
|---------------------------------------------------------------------------------------------------------------------------|--|
| <div><div></div><div></div><div></div></div>                                                                              |  |
| <div><div></div><div></div><div></div><div></div><div></div></div>                                                        |  |
| <div><div></div><div></div><div></div><div></div><div></div><div></div><div></div><div></div><div></div><div></div></div> |  |
| Sem1 Sem2 Sem3 Sem4                                                                                                       |  |
| 94% 92% 95% 94%                                                                                                           |  |
|                                                                                                                           |  |
| Performance por Técnico                                                                                                   |  |
| João (28 tickets)   SLA 96%   Satisfação 9.2                                                                              |  |
| Maria (24)   SLA 91%   Satisfação 8.7                                                                                     |  |
| Pedro (19)   SLA 89%   Satisfação 8.3                                                                                     |  |
| Ana (31)   SLA 94%   Satisfação 9.0                                                                                       |  |

Dashboard Técnico (Suporte)

|                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------|
| Minha Fila - 15 tickets                                                                                                                |
| P1 (2) <div><div></div><div></div></div>   P2 (5) <div><div></div><div></div><div></div><div></div><div></div></div>   P3 (6)   P4 (2) |
| Próximo Vencimento SLA:                                                                                                                |
| #12543 - Wi-Fi não conecta (3h 15m) [CRÍTICO]                                                                                          |
| #12541 - Reset senha (5h 45m) [OK]                                                                                                     |
| #12532 - Instalar VS Code (18h) [OK]                                                                                                   |
| Últimas Resoluções:                                                                                                                    |
| ✓ #12540 - Liberado acesso (45m ago)                                                                                                   |
| ✓ #12539 - Configurado Outlook (2h ago)                                                                                                |
| ✓ #12538 - Trocado HD (yesterday)                                                                                                      |

Relatório Customizável

Filtros Disponíveis:

- Período: [data inicial] [data final]
- Tipo: [Incidente] [Requisição] [Problema] [Mudança]
- Prioridade: [P1] [P2] [P3] [P4]
- Status: [Novo] [Aberto] [Resolvido] [Fechado] [Cancelado]
- Categoria: [Dropdown multiseleção]
- Técnico: [Dropdown multiseleção]
- Grupo: [Dropdown multiseleção]
- SLA: [OK] [Crítico] [Expirado]

Colunas (selecionáveis):

- Número do Ticket
- Título
- Prioridade
- Status
- Técnico Atribuído
- Data Abertura
- Data Fechamento
- Tempo Resolução

- └─ SLA Status
- └─ Satisfação (rating)
- └─ Comentários

Exportação:

- └─ CSV (importável em Excel/Sheets)
- └─ Excel (XLSX com formatação)
- └─ PDF (relatório formatado)
- └─ JSON (para integração)

### Gráficos Disponíveis

- **Pizza:** Distribuição por prioridade, status, categoria
- **Barra:** Tickets por técnico, por grupo, por mês
- **Linha:** Tendência SLA, satisfação, backlog ao longo do tempo
- **Heatmap:** Volume por hora do dia (identificar picos)

#### 3.9.3 Agendamento de Relatórios

## Admin pode agendar envio automático de relatório

```
@dataclass
class ScheduledReport:
 name: str
 query: ReportQuery # Filtros
 schedule: str # cron: "0 09 * * MON" (todo seg 09:00)
 recipients: List[str] # E-mails
 format: str # 'csv', 'excel', 'pdf'
```

## Exemplo: CEO recebe resumo executivo toda segunda-feira às 09:00

#### 3.9.4 Tecnologias Específicas

- **Chart.js ou Plotly:** Renderização de gráficos
- **Pandas + Openpyxl:** Exportação para Excel
- **WeasyPrint ou ReportLab:** Geração de PDF
- **Celery:** Agendamento de relatórios

#### 3.9.5 Endpoints e Operações

| Operação                 | Método | Endpoint                   | Descrição             |
|--------------------------|--------|----------------------------|-----------------------|
| Dashboard Executivo      | GET    | /api/dashboards/executive  | KPIs gerenciais       |
| Dashboard Técnico        | GET    | /api/dashboards/technician | Fila pessoal          |
| Gerar Relatório          | POST   | /api/reports/generate      | Com filtros           |
| Listar Relatórios Salvos | GET    | /api/reports               | Relatórios anteriores |
| Exportar                 | GET    | /api/reports/{id}/export   | CSV/Excel/PDF         |
| Agendar Relatório        | POST   | /api/reports/schedule      | Admin                 |

### 3.10 Auditoria e Segurança

#### 3.10.1 Descrição Geral

Registra todas as ações críticas, logins e mudanças. Essencial para compliance, forensics e conformidade.

#### 3.10.2 Componentes Principais

##### Audit Log

```
CREATE TABLE audit_logs (
 id UUID PRIMARY KEY,
 action ENUM (
 'LOGIN', 'LOGOUT', 'CREATE', 'UPDATE', 'DELETE',
 'APPROVE', 'REJECT', 'ESCALATE', 'ASSIGN'
),
 resource_type ENUM (
 'TICKET', 'ASSET', 'USER', 'CI', 'KNOWLEDGE', 'SLA', ...
),
 resource_id UUID, -- ID do recurso afetado
 user_id UUID NOT NULL (FK users.id),
```

```
-- Before/After
before JSON, -- Estado anterior (se UPDATE)
after JSON, -- Estado novo
```

```
-- Contexto
ip_address INET,
user_agent VARCHAR(500),
timestamp TIMESTAMP,

-- Severidade
severity ENUM ('INFO', 'WARNING', 'CRITICAL')
```

);

### Exemplos de Audit Log

1. LOGIN (INFO)  
user\_id: u123  
ip\_address: 200.123.45.67  
user\_agent: Mozilla/5.0...  
timestamp: 2026-01-26 09:15:00
2. UPDATE ticket #12345 (INFO)  
user\_id: u456 (técnico)  
before: {status: 'NOVO', assigned\_to: null}  
after: {status: 'EM\_ANDAMENTO', assigned\_to: 'u456'}  
timestamp: 2026-01-26 09:16:30
3. DELETE asset a789 (CRITICAL)  
user\_id: u123 (admin)  
before: {name: 'Monitor Dell', serial: 'XYZ123', ...}  
after: null  
timestamp: 2026-01-26 10:00:00  
// Notifica admin sobre exclusão

### Segurança de Senha

- **Hashing:** Bcrypt com salt de 12 rounds (apenas admin local, LDAP preferred)
- **Requisitos:**
  - Mínimo 12 caracteres
  - Letras maiúsculas, minúsculas, números, símbolos
  - Sem padrões óbvios (123456, QWERTY, etc)
  - Validação contra lista de senhas comuns (rockyou.txt, etc)
- **Expiração:** 90 dias, obrigando troca periódica
- **Histórico:** Impede reutilização de últimas 5 senhas

### Validação CSRF



# Todos os POST/PUT/DELETE endpoints exigem CSRF token

## Frontend:

```
// 1. Obter token na sessão
const csrfToken = document.querySelector('meta[name="csrf-token"]').content

// 2. Enviar em header
fetch('/api/tickets', {
 method: 'POST',
 headers: {
 'X-CSRF-Token': csrfToken,
 'Content-Type': 'application/json'
 },
 body: JSON.stringify(...)
})
```

## Backend:

```
@app.post("/api/tickets")
async def create_ticket(request: Request, data: TicketCreate):
 csrf_token = request.headers.get('X-CSRF-Token')
 if not validate_csrf(csrf_token, request.session):
 raise HTTPException(403, "CSRF validation failed")
```

### Rate Limiting

## Implementar rate limiting por IP/usuário

```
@limiter.limit("5/minute") # 5 requisições por minuto
@app.post("/api/auth/login")
async def login(credentials: LoginRequest):
 # Protege contra brute force
 pass

@limiter.limit("100/minute") # Limite mais permissivo para usuários
@app.get("/api/tickets")
async def list_tickets():
 pass
```

### IP Logado

- Cada login registra IP e User-Agent
- Notificação se IP desconhecido tenta acessar
- Dashboard mostra "Sessões ativas" com IP e hora

### 3.10.3 Conformidade Regulatória

- **LGPD:** Direito ao esquecimento (anonymize user data após X anos)
- **Retenção de Logs:** Auditoria por 1 ano, backups por 3 anos
- **Criptografia em Trânsito:** HTTPS obrigatório
- **Criptografia em Repouso:** Senhas com bcrypt, dados sensíveis com AES-256

### 3.10.4 Tecnologias Específicas

- **Passlib + Bcrypt:** Hashing de senha
- **python-dotenv:** Variáveis de ambiente (API keys, secrets)
- **slowapi:** Rate limiting baseado em IP
- **cryptography:** Criptografia de dados sensíveis

### 3.10.5 Endpoints e Operações

| Operação          | Método | Endpoint                                | Descrição              |
|-------------------|--------|-----------------------------------------|------------------------|
| Listar Audit Logs | GET    | /api/admin/audit-logs                   | Admin apenas           |
| Filtrar Logs      | GET    | /api/admin/audit-logs/search            | Por usuário, ação, etc |
| Sessões Ativas    | GET    | /api/admin/sessions                     | IPs, horários          |
| Revogar Sessão    | POST   | /api/admin/sessions/{session_id}/revoke | Logout forçado         |
| Historico Login   | GET    | /api/security/login-history             | Pessoal                |

---

## 3.11 Notificações e Integrações

### 3.11.1 Descrição Geral

Sistema de notificações que mantém usuários informados sobre tickets, aprovações, SLA crítico. Suporta múltiplos canais.

### 3.11.2 Componentes Principais

#### Tipos de Notificações

| Evento                       | Destinatário         | Mensagem                                                      |
|------------------------------|----------------------|---------------------------------------------------------------|
| <b>Ticket Aberto</b>         | Técnico atribuído    | "Novo ticket #12345: Wi-Fi não conecta (P2)"                  |
| <b>Comentário Adicionado</b> | Partes interessadas  | "João comentou em #12345: Wi-Fi volta em 1h"                  |
| <b>SLA Crítico</b>           | Técnico, Manager     | "ALERTA: Ticket #12345 em SLA crítico (70%)"                  |
| <b>Ticket Resolvido</b>      | Solicitante          | "Seu ticket #12345 foi resolvido, clique aqui para confirmar" |
| <b>Requisição Aprovada</b>   | Solicitante          | "Sua requisição de VS Code foi aprovada!"                     |
| <b>Requisição Rejeitada</b>  | Solicitante          | "Sua requisição de VS Code foi rejeitada (motivo: custos)"    |
| <b>Atribuição Mudada</b>     | Técnico antigo, novo | "Ticket #12345 reatribuído para você"                         |

### Canais de Notificação

```
@dataclass
class NotificationChannel:
 name: str
 enabled: bool
```

```
Implementações específicas
```

```
class EmailNotification(NotificationChannel):
 smtp_host: str = "smtp.coppead.ufrj.br"
 smtp_port: int = 587
 from_email: str = "suporte-ti@coppead.ufrj.br"
```

```
 async def send(self, recipient: User, subject: str, body: str):
 # Renderiza template com CSS inlined
 # Envia via SMTP (Celery async)
 pass
```

```
class SlackNotification(NotificationChannel):
 webhook_url: str # Configurável
```

```
async def send(self, recipient: User, message: str):
 # Envia para canal Slack
 # Ex: #suporte-ti ou DM
 pass
```

```
class WebNotification(NotificationChannel):
 """Push notification no navegador + badge"""
```

```
async def send(self, recipient: User, message: str):
 # Broadcast via WebSocket
 # Salva em DB para histórico
 pass
```

## Futuro:

```
class TeamsNotification(NotificationChannel):
 pass
```

```
class SMSNotification(NotificationChannel):
 pass
```

### Template de E-mail

## Notificação de Suporte - COPPEAD/UFRJ

Olá {{ user.first\_name }},

{{ message }}

**Ticket #{{ ticket.number }}**

Título: {{ ticket.title }}

Prioridade: {{ ticket.priority }}

Status: {{ ticket.status }}

[Ver Ticket](#)

Este é um e-mail automático. Não responda a este endereço.

© 2026 COPPEAD/UFRJ. Todos os direitos reservados.

### Preferências de Notificação

Cada usuário pode configurar:

@dataclass

class NotificationPreferences:

user\_id: UUID

```
Por tipo de evento
notify_on_ticket_assigned: bool = True
notify_on_comment: bool = True
notify_on_sla_critical: bool = True
notify_on_resolution: bool = True
notify_on_approval_status: bool = True

Por canal
email_enabled: bool = True
slack_enabled: bool = False
web_push_enabled: bool = True

Horários quiet (não enviar notificações)
quiet_hours_enabled: bool = True
quiet_hours_start: time = Time(18, 0) # 18:00
quiet_hours_end: time = Time(08, 0) # 08:00 (next day)
```

### Fila de Notificações (Celery)

## Ao ocorrer evento, enfileira notificação (async)

```
@app.post("/api/tickets")
async def create_ticket(data: TicketCreate, current_user: User):
 ticket = await db.tickets.insert(data)
```

```
Enfileira notificação
send_ticket_created_notification.delay(
 ticket_id=ticket.id,
 assigned_to_user_id=assigned_technician.id
)

return ticket
```

# Celery worker processa fila

```
@celery_app.task
def send_ticket_created_notification(ticket_id: UUID, assigned_to_user_id: UUID):
 ticket = get_ticket(ticket_id)
 user = get_user(assigned_to_user_id)

 if user.preferences.email_enabled:
 send_email(
 to=user.email,
 subject=f"Novo ticket: {ticket.title}",
 body=render_template("ticket_created.html", ticket=ticket, user=user)
)

 if user.preferences.slack_enabled:
 send_slack_message(
 webhook=user.slack_webhook,
 message=f"Novo ticket #{ticket.number}: {ticket.title}"
)
```

## 3.11.3 Webhooks Customizáveis

Permite integrações externas (GitHub, Zabbix, etc) criarem tickets:

# Admin cadastra webhook

```
@dataclass
class WebhookConfig:
 id: UUID
 name: str = "GitHub Push Webhook"
 url: str = "https://github.com/..."
 event_type: str = "push"

 action: str = "create_ticket"
 ticket_template: Dict = {
 "category": "Development",
 "priority": "P3",
 "title": "GitHub: {{ repo }} - {{ commit_message }}"
 }
```

```
is_active: bool = True
```

Quando GitHub envia webhook:

POST /api/webhooks/github

```
{
 "action": "push",
 "repository": "glpi-system",
 "commit_message": "Fix: SLA calculation",
 ...
}
```

Sistema cria ticket com template acima

#### 3.11.4 Tecnologias Específicas

- **Celery + Redis:** Fila distribuída de notificações
- **Jinja2:** Renderização de templates
- **smtplib / email.mime:** Envio SMTP
- **requests:** Chamadas HTTP para Slack/Teams
- **WebSocket / [Socket.io](#):** Notificações em tempo real

#### 3.11.5 Endpoints e Operações

| Operação                 | Método  | Endpoint                               | Descrição |
|--------------------------|---------|----------------------------------------|-----------|
| Preferências Notificação | GET/PUT | /api/users/me/notification-preferences | Usuário   |
| Histórico Notificações   | GET     | /api/users/me/notifications            | Pessoal   |
| Marcar como Lido         | POST    | /api/notifications/{id}/read           | Mark read |
| Histórico Webhook        | GET     | /api/admin/webhooks/history            | Admin     |
| Reenviar Notificação     | POST    | /api/admin/notifications/{id}/resend   | Admin     |

### 3.12 Experiência do Usuário e Automação

#### 3.12.1 Descrição Geral

Recursos que simplificam uso e reduzem tarefas manuais.

#### 3.12.2 Componentes Principais

##### Sugestão de Artigos KB

- Ao digitar título do ticket, algoritmo TF-IDF identifica artigos relevantes
- Mostrado em sidebar "Esses artigos podem ajudar..."
- Se usuário seleciona artigo como solução, reduz necessidade de chamado

##### Auto-atribuição Inteligente

def auto\_assign\_ticket(ticket: Ticket) -> User:

"""

1. Identifica grupo técnico correto pela categoria
2. Lista técnicos ativos no grupo
3. Calcula carga de cada técnico:
  - Tickets abertos
  - SLA crítico / expirado (peso maior)
  - Experiência na categoria (histórico)
4. Atribui para técnico com menor carga + melhor experiência

"""

```
Exemplo:
Ticket categoria "Email"
Grupo: TI-Messaging
```



```
Técnicos: João (8 tickets), Maria (5), Pedro (6)
João tem 2 P1 críticos → maior peso
Atribui para Maria (menor carga)
```

## Auto-encerramento de Tickets Inativos

# Celery job diário

```
@periodic_task(run_every=crontab(hour=1, minute=0))
def auto_close_inactive_tickets():
 """
 1. Encontra tickets RESOLVIDO há mais de 7 dias sem feedback
 2. Se nenhuma atividade: fecha automaticamente
 3. Notifica usuário: "Seu ticket foi fechado por inatividade"
 """
```

## Templates de Resposta

Técnico pode usar templates pré-definidos para responder:

Categoria: Email

Template 1: "Resetar Senha"

"Olá {{ user.first\_name }},

Conforme solicitado, sua senha foi resetada.

Código temporário: {{ temp\_password }}

Você pode fazer login em {{ login\_url }} e alterar para uma senha definitiva.

Obrigado,  
Suporte TI"

Template 2: "Acesso Concedido"

"Olá {{ user.first\_name }},

Seu acesso a {{ resource }} foi concedido com sucesso.

Você pode acessar em {{ access\_url }}

Qualquer dúvida, estou à disposição."

## Formulários Condicionais

Catálogo pode ter perguntas que aparecem/desaparecem conforme respostas anteriores:

```
{
 "form_schema": {
 "properties": {
 "request_type": {
 "enum": ["Instalação", "Suporte", "Remoção"],
 "title": "Tipo de Solicitação"
 },
 },
 },
}
```

```

"software_name": {
 "type": "string",
 "title": "Nome do Software"
},
"installation_target": {
 "type": "string",
 "title": "Computador de destino",
 "visible_when": "request_type === 'Instalação'" // Apareça apenas se instalação
},
"removal_reason": {
 "type": "string",
 "title": "Motivo da Remoção",
 "visible_when": "request_type === 'Remoção'"
}
}
}
}
}

```

### Busca Rápida com Autocomplete

Input: "wifi"

Sugestões aparecem:

1. "Wi-Fi não conecta (FAQ)" → abre artigo
2. "#12341 - Problema Wi-Fi corporativo" → abre ticket
3. "Novo ticket: Wi-Fi" → abre formulário
4. "Wi-Fi 6802.11ax" → procura em CI/assets

#### 3.12.3 Tecnologias Específicas

- **React Query**: Cache inteligente para dados
- **Debounce**: Reduz chamadas API durante digitação
- **Elasticsearch**: Busca full-text com autocomplete
- **Algorithms**: ML para priorização (futuro)

---

### 3.13 Central de Aprovações

#### 3.13.1 Descrição Geral

Painel centralizado para gestores aprovarem requisições e mudanças. Suporta múltiplos níveis hierárquicos.

#### 3.13.2 Componentes Principais

##### Fluxo de Aprovação

Solicitação criada (ex: Instalar Software)

↓

Sistema cria approval\_request

↓

Gerente recebe notificação: "Aguardando aprovação de João"

↓

Gerente revisa requisição:

- Dados da solicitação
  - Justificativa
  - Impacto / Custo (se aplicável)
- ↓
- Aprova ou Rejeita (com comentário obrigatório se rejeitar)
- ↓
- Se aprovado → Executa (cria subtasks)
- Se rejeitado → Volta para solicitante com motivo

## Múltiplos Níveis

Alguns serviços exigem aprovação em cascata:

Requisição: Compra de Monitor (custo R\$2000)

Nível 1: Gerente de Departamento

└─ Aprova/Rejeita

Se aprovado no Nível 1:

└─ Nível 2: Diretor Administrativo

└─ Aprova/Rejeita

|

└─ Se aprovado no Nível 2:

└─ Nível 3: CFO

└─ Aprova/Rejeita

Apenas se todos aprovarem → Executa

## Estrutura de Dados

```
CREATE TABLE approval_requests (
 id UUID PRIMARY KEY,
 request_id UUID NOT NULL (FK service_requests.id),
```

```
 level INT, -- Nível hierárquico (1, 2, 3...)
 required_approver_count INT, -- Quantos devem aprovar?

 status ENUM ('PENDENTE', 'APROVADO', 'REJEITADO'),

 created_at TIMESTAMP,
 expires_at TIMESTAMP -- Se não responder em 3 dias, escala
```

```
);
```

```
CREATE TABLE approval_decisions (
 id UUID PRIMARY KEY,
 approval_request_id UUID NOT NULL (FK approval_requests.id),
 approver_id UUID NOT NULL (FK users.id),
```

```
decision ENUM ('APROVADO', 'REJEITADO'),
comment TEXT, -- Obrigatório se rejeitar
```

```
decided_at TIMESTAMP
```

```
);
```

## Dashboard de Aprovações

|                                       |
|---------------------------------------|
| Minhas Aprovações Pendentes           |
| Aguardando minha decisão: 7           |
| Aguardando outras pessoas: 3          |
|                                       |
| PENDENTES:                            |
| 1. [P2] Instalar VS Code - João Silva |
| Justificativa: Desenvolvimento Python |
| [Aprovar] [Rejeitar] [Ver Mais]       |
|                                       |
| 2. [P3] Reset Senha - Maria Santos    |
| [Aprovar] [Rejeitar] [Ver Mais]       |
|                                       |
| 3. [P1] Acesso VPN - Pedro Costa      |
| CRÍTICO - Aguarda há 5 horas!         |
| [Aprovar] [Rejeitar] [Ver Mais]       |
|                                       |
| HISTÓRICO (últimas ações):            |
| ✓ Aprovado: Notebook novo - 2h atrás  |
| ✗ Rejeitado: Software X - 1 dia atrás |
| Motivo: Conflito com política de TI   |

## Escalonamento Automático

# Celery job: verifica aprovações expiradas

```
@periodic_task(run_every=crontab(minute='*/15')) # A cada 15 minutos
```

```
def escalate_expired_approvals():
```

```
"""
```

```
1. Encontra approval_requests expiradas (3 dias)
```

```
2. Se ainda PENDENTE:
```

```
- Sobe para próximo nível (se houver)
```

```
- Notifica gerente superior
```

```
- Ou aprova automaticamente (conforme política)
```

```
"""
```

### 3.13.3 Endpoints e Operações

| Operação          | Método | Endpoint                             | Descrição         |
|-------------------|--------|--------------------------------------|-------------------|
| Listar Pendentes  | GET    | /api/approvals/pending               | Pessoal           |
| Detalhe Aprovação | GET    | /api/approvals/{approval_id}         | Dados completos   |
| Aprovar           | POST   | /api/approvals/{approval_id}/approve | Decisão           |
| Rejeitar          | POST   | /api/approvals/{approval_id}/reject  | Com comentário    |
| Histórico         | GET    | /api/approvals/history               | Histórico pessoal |

---

### 3.14 a 3.19 - Módulos Restantes (Resumidos)

Pelos limites de espaço, apresento resumo dos módulos finais. Cada um seguiria a mesma estrutura (descrição, componentes, dados, endpoints):

#### 3.14 Licenciamento e Contratos

- Registro de software com datas de expiração
- Contratos de manutenção por asset
- Alertas automáticos 30 dias antes do vencimento
- Dashboard de licenças vencidas/próximas

#### 3.15 Feedback e Satisfação

- Pesquisa pós-ticket (1-5 estrelas)
- Cálculo de NPS (Net Promoter Score)
- Dashboard por técnico, categoria, período

#### 3.16 Anúncios no Portal

- Avisos agendados (ex: manutenção programada)
- Visibilidade por grupo
- Destaque em homepage

#### 3.17 Calendário de Horário Útil

- Feriados personalizados
- Horários variáveis por departamento (TI 07:00-20:00, Administrativo 08:00-17:00)
- Sincronização com SLA

#### 3.18 API Pública e Webhooks

- Endpoints públicos para criação de tickets (integração)
- API Keys com permissões granulares
- Webhooks enviados para URLs externas (GitHub, Zabbix)

### 3.19 Tags e Classificação Avançada

- Tags aplicáveis a tickets, ativos, artigos, serviços
- Busca por tag
- Sugestão automática baseada em ML

---

## 4. Fluxos Principais do Sistema

### 4.1 Fluxo de Incidente

---

#### FLUXO DE INCIDENTE |

---

##### 1. ABERTURA DO TICKET

- Portal: Usuário preenche formulário
  - Título: "Wi-Fi não conecta"
  - Descrição: "Não consigo me conectar ao Wi-Fi desde as 9h"
  - Categoria: Redes
  - Asset: Notebook Dell (UFRJ-PC-12345)
- E-mail: [Suporte@coppead.ufrj.br](mailto:Suporte@coppead.ufrj.br) (parser IMAP)
- Sistema cria ticket automaticamente

##### 2. TRIAGEM AUTOMÁTICA

- Calcula prioridade: Urgência × Impacto
  - Urgência: MÉDIA (1 usuário afetado)
  - Impacto: INDIVIDUAL
  - Prioridade: P3 (MÉDIO)
- Associa SLA: 24 horas para resolução
- Busca KB: "Problema Wi-Fi"
- Sugere: "Como reconectar ao Wi-Fi" (92% match)

##### 3. SUGESTÃO AO USUÁRIO

- "Esses artigos podem resolver seu problema:"
  - Como reconectar ao Wi-Fi corporativo
  - Problemas comuns de rede
- Não resolveu? Clique 'Continuar com Ticket'

##### 4. ATRIBUIÇÃO

- Análise: Categoria "Redes" → Grupo "TI-Infra"
- Cálculo de carga:
  - João (8 tickets, 2 P1 críticos) → peso alto
  - Maria (5 tickets) → peso baixo
  - Pedro (6 tickets) → peso médio

- └─ Atribui: Maria (menor carga)
- └─ Maria recebe notificação
- 5. INVESTIGAÇÃO
  - └─ Maria adiciona comentário INTERNO:
    - └─ "Usuário em que prédio? VPN ativa? Driver atualizado?"
  - └─ Ticket volta para AGUARDANDO
    - └─ SLA pausado (não conta tempo esperando resposta)
  - └─ Usuário responde:
    - └─ "Prédio B, sem VPN, driver Windows atualizado"
  - └─ Maria identifica problema: Problema DHCP no access point B
- 6. RESOLUÇÃO
  - └─ Maria reinicia AP do Prédio B
  - └─ Verifica: Usuário consegue conectar
  - └─ Adiciona comentário PÚBLICO:
    - └─ "Wi-Fi estava sem DHCP. Reiniciei access point. Agora está funcionando. Confirme se conecta."
  - └─ Marca ticket: RESOLVIDO
- 7. CONFIRMAÇÃO DO USUÁRIO
  - └─ Usuário clica "Confirmar resolução"
  - └─ Sistema marca: FECHADO
  - └─ Envia pesquisa de satisfação:
    - └─ "Como foi nosso atendimento? 1-5 ★"
- 8. FEEDBACK
  - └─ Usuário deixa: 5 ★ + "Rápido e eficiente!"
  - └─ Sistema atualiza NPS de Maria
- 9. ENCERRAMENTO
  - └─ Ticket arquivado
  - Histórico preservado
  - Tempo resolução: 1h 45m
  - SLA cumprido: ✓

## 4.2 Fluxo de Requisição

1. SELEÇÃO DE SERVIÇO
  - └─ Usuário acessa Portal → Catálogo → "Instalar Software"
2. PREENCHIMENTO DO FORMULÁRIO
  - └─ Nome do Software: Visual Studio Code
  - └─ Justificativa: "Desenvolvimento em Python"
  - └─ Computador: UFRJ-PC-12345 (seleção de seus assets)
  - └─ Validações em tempo real
3. REVISÃO
  - └─ Resumo: "Solicitar instalação de VS Code em UFRJ-PC-12345"
  - └─ [Enviar] [Cancelar]
4. ABERTURA DE REQUISIÇÃO
  - └─ Sistema cria ticket tipo REQUISIÇÃO

- └─ Status: NOVO → TRIAGEM
- └─ Gera tarefas automáticas:
  - Task 1: Validar licença (TI-Software)
  - Task 2: Instalar (TI-Deskside)
- 5. APROVAÇÃO (se requerida)
  - └─ Gerente recebe notificação
  - └─ Aprova → Segue para execução
- 6. EXECUÇÃO
  - └─ TI-Software valida licença (Celery notifica)
  - └─ TI-Deskside instala no computador
    - Conecta remoto ao PC
    - Download + instalação
    - Valida instalação
  - └─ Ambas tarefas marcadas CONCLUÍDO
- 7. ENCERRAMENTO
  - └─ Sistema marca requisição: RESOLVIDO
  - └─ Notifica usuário: "VS Code foi instalado!"
  - └─ Pede feedback
- 8. FEEDBACK
  - └─ Usuário avalia: 5 ★

### 4.3 Fluxo de Problema + Mudança

1. ABERTURA DE PROBLEMA
  - └─ Técnico identifica: 15 tickets sobre Wi-Fi em 2 horas
  - └─ Cria ticket tipo PROBLEMA:
    - Título: "Falha DHCP em access point Prédio B"
    - RCA: "Servidor DHCP sem memória, descartou clientes"
    - Workaround: "Use IP estático 192.168.X.X até solução"
  - └─ Associa 15 incidentes ao problema
2. CRIAÇÃO DE MUDANÇA (RFC)
  - └─ Descreve a mudança necessária:
    - Upgrade DHCP Server (versão X → Y)
    - Impacto: 2 horas de downtime
    - Janela: Domingo 02:00-04:00
    - Rollback: Reverter para versão anterior
  - └─ Submete para CAB (Change Advisory Board)
3. AVALIAÇÃO
  - └─ CAB revisa mudança
  - └─ Avalia risco, complexidade, recursos
  - └─ Aprova com ressalvas: "Testado antes?"
4. APROVAÇÃO
  - └─ Gerente aprova RFC
  - └─ Sistema agenda para domingo 02:00



## 5. EXECUÇÃO

- └─ Técnico conecta ao servidor
- └─ Documenta estado atual (screenshot, versão)
- └─ Executa backup
- └─ Para serviço DHCP
- └─ Upgrade pacote
- └─ Inicia serviço
- └─ Valida funcionamento
- └─
- └─ Adiciona comentários de progresso

## 6. VALIDAÇÃO PÓS-MUDANÇA

- └─ Técnico confirma:  
"Servidor DHCP respondendo normalmente,  
clientes recebendo IPs em <50ms"
- └─
- └─ Marca change como VALIDADO

## 7. REVISÃO FINAL

- └─ CAB faz post-mortem:  
"Alteração bem-sucedida, sem problemas"
- └─ Documenta lições aprendidas
- └─
- └─ Fecha change request

## 8. RESOLUÇÃO DO PROBLEMA

- └─ Problema marcado ENCERRADO
- RCA implementada
- Prazo: 3 dias
- 15 incidentes resolvidos

---

## ▮ 5. Exemplo de Caso de Uso Completo

### Cenário: Novo Colaborador - Onboarding

SEGUNDA-FEIRA 08:00

- └─ RH abre requisição "Onboarding: João Silva (novo analista)"
- └─ Formulário solicita:
  - Nome completo
  - CPF
  - Departamento
  - Data de início
  - Cargo
- └─
- └─ Submete para aprovação

SEGUNDA-FEIRA 09:00

- └─ Gerente aprova requisição
- └─ Sistema cria tarefas automáticas:
  1. [TI-Systems] Criar conta AD
  2. [TI-Messaging] Criar e-mail e mailbox Exchange
  3. [TI-Security] Adicionar a grupos de segurança
  4. [TI-Deskside] Provisionar notebook + periféricos

5. [TI-Training] Agendar treinamento de sistemas

6. [Facilities] Preparar mesa + chave de acesso

SEGUNDA-FEIRA 09:30

- └ TI-Systems recebe notificação
- └ Cria conta AD "jsilva"
- └ Define senha temporária: P@ssw0rd123!
- └ Marca task como CONCLUÍDO
- └ Sistema notifica TI-Messaging (dependência)

SEGUNDA-FEIRA 10:00

- └ TI-Messaging cria mailbox
- └ Configura encaminhamento
- └ Adiciona a listas de distribuição
- └ Marca task como CONCLUÍDO
- └ Sistema notifica TI-Security (dependência)

SEGUNDA-FEIRA 10:30

- └ TI-Security adiciona a grupos:
  - └ "Colaboradores"
  - └ "Acesso Wi-Fi"
  - └ "Acesso VPN"
  - └ "Departamento-Análise"
- └ Marca task como CONCLUÍDO

SEGUNDA-FEIRA 11:00

- └ TI-Deskside provisiona notebook
- └ Instala Windows 11 + drivers
- └ Instala Office 365
- └ Instala software padrão (VLC, 7zip, AnyDesk)
- └ Configura impressoras padrão
- └ Instala certificados SSL
- └ Entrega para RH repassar para João

SEGUNDA-FEIRA 14:00

- └ TI-Training agenda "Induction Training"
- └ Apresentação do sistema de TI
- └ Credenciais e segurança
- └ Como abrir tickets de suporte
- └ Cria evento no calendário de João

TERÇA-FEIRA 08:00

- └ João faz login pela primeira vez
- └ Sistema força alteração de senha (política)
- └ Mostra guia de onboarding
- └ Portal exibe:
  - "Bem-vindo ao Sistema de Suporte COPPEAD/UFRJ"
  - Links úteis
  - Próximas tarefas

- | - Base de Conhecimento recomendada
- |
- └─ João consegue acessar todos os sistemas

TERÇA-FEIRA 14:00

- └─ TI-Training realiza treinamento
- └─ João aprende a abrir tickets e acompanhar

Resultado: Onboarding de novo colaborador automático, coordenado e sem necessidade de e-mails manuais!

---

## ▮ 6. Infraestrutura e DevOps

### 6.1 Arquitetura de Containers (Docker Compose)

version: '3.8'

services:

## Frontend (React SPA)

frontend:

build:

context: ./frontend

dockerfile: Dockerfile

ports:

- "3000:3000"

volumes:

- ./frontend:/app

environment:

- REACT\_APP\_API\_URL=http://api:8000

- REACT\_APP\_WS\_URL=ws://api:8000/ws

depends\_on:

- api

## Backend (FastAPI)

api:

build:

context: ./backend

dockerfile: Dockerfile

ports:

- "8000:8000"

volumes:

- ./backend:/app

environment:

- DATABASE\_URL=postgresql://user:pass@db:5432/itsm

- REDIS\_URL=redis://redis:6379

- LDAP\_SERVER=ldap://ldap.coppead.ufrj.br:389

- SECRET\_KEY=\${SECRET\_KEY}
- ENVIRONMENT=development

depends\_on:

- db
- redis
- ldap

## Banco de Dados

db:

image: postgres:15-alpine

ports:

- "5432:5432"

environment:

- POSTGRES\_DB=itsm
- POSTGRES\_USER=itsm\_user
- POSTGRES\_PASSWORD=\${DB\_PASSWORD}

volumes:

- postgres\_data:/var/lib/postgresql/data
- ./backend/sql/init.sql:/docker-entrypoint-initdb.d/init.sql

## Cache e Sessões

redis:

image: redis:7-alpine

ports:

- "6379:6379"

volumes:

- redis\_data:/data

## Worker Celery (Background Jobs)

celery\_worker:

build:

context: ./backend

dockerfile: Dockerfile.celery

environment:

- DATABASE\_URL=postgresql://user:pass@db:5432/itsm
- REDIS\_URL=redis://redis:6379
- CELERY\_BROKER\_URL=redis://redis:6379/0
- CELERY\_RESULT\_BACKEND=redis://redis:6379/1

depends\_on:

- db
- redis
- api

# Celery Beat (Scheduler)

celery\_beat:  
build:  
context: ./backend  
dockerfile: Dockerfile.celery  
command: celery -A tasks beat --loglevel=info  
environment:  
- DATABASE\_URL=postgresql://user:pass@db:5432/itsm  
- REDIS\_URL=redis://redis:6379  
- CELERY\_BROKER\_URL=redis://redis:6379/0  
- CELERY\_RESULT\_BACKEND=redis://redis:6379/1  
depends\_on:  
- db  
- redis

# Reverse Proxy (Nginx)

nginx:  
image: nginx:alpine  
ports:  
- "80:80"  
- "443:443"  
volumes:  
- ./nginx.conf:/etc/nginx/nginx.conf  
- ./ssl:/etc/nginx/ssl  
depends\_on:  
- api  
- frontend

# LDAP (Mock para desenvolvimento)

ldap:  
image: osixia/openldap:latest  
ports:  
- "389:389"  
environment:  
- LDAP\_ORGANISATION=COPPEAD  
- LDAP\_DOMAIN=[coppead.ufrj.br](http://coppead.ufrj.br)  
- LDAP\_ADMIN\_PASSWORD=\${LDAP\_PASSWORD}  
  
volumes:  
postgres\_data:  
redis\_data:

## 6.2 CI/CD com GitHub Actions

# **.github/workflows/deploy.yml**

name: Build, Test & Deploy

on:

push:

branches: [main, develop]

pull\_request:

branches: [main, develop]

jobs:

test:

runs-on: ubuntu-latest

services:

postgres:

image: postgres:15

env:

POSTGRES\_PASSWORD: postgres

options: >-

--health-cmd pg\_isready

--health-interval 10s

--health-timeout 5s

--health-retries 5

redis:

image: redis:7

options: >-

--health-cmd "redis-cli ping"

--health-interval 10s

--health-timeout 5s

--health-retries 5

steps:

- uses: actions/checkout@v3

- name: Set up Python

uses: actions/setup-python@v4

with:

python-version: '3.11'

- name: Install dependencies

run: |

```
python -m pip install --upgrade pip
pip install -r backend/requirements.txt
```

- name: Run linting (Flake8)

run: |

```
flake8 backend/app --count --select=E9,F63,F7,F82 --show-source --statistics
flake8 backend/app --count --exit-zero --max-complexity=10 --max-line-length
```

- name: Run tests

run: |

```
pytest backend/tests -v --cov=backend/app --cov-report=xml
```

- name: Upload coverage

uses: codecov/codecov-action@v3

with:

file: ./coverage.xml

- name: Build frontend

run: |

```
cd frontend
npm install
npm run build
```

- name: Build Docker images

run: |

```
docker build -t itism-api:${{ github.sha }} ./backend
docker build -t itism-frontend:${{ github.sha }} ./frontend
```

- name: Push to registry

if: github.ref == 'refs/heads/main'

run: |

```
echo ${ secrets.DOCKER_PASSWORD } | docker login -u ${ secrets.DOCKER_USERNAME }
docker tag itism-api:${{ github.sha }} docker.io/${ secrets.DOCKER_USERNAME }/itism-api:latest
docker push docker.io/${ secrets.DOCKER_USERNAME }/itism-api:latest
```

```
- name: Deploy to production
 if: github.ref == 'refs/heads/main'
 run: |
 ssh-keyscan -H ${ secrets.DEPLOY_HOST } >> ~/.ssh/known_hosts
 ssh -i ~/.ssh/deploy_key user@${ secrets.DEPLOY_HOST } << EOF
 cd /app/itsm
 docker-compose pull
 docker-compose up -d
 docker-compose exec -T api alembic upgrade head
 EOF
```

### 6.3 Backup e Disaster Recovery

```
#!/bin/bash
```

## backup.sh - Backup automático diário

```
BACKUP_DIR="/backups"
DATE=$(date + DB_BACKUP="//BACKUP_DIR/db_DATE.sql.gz"APP_BACKUP="//
BACKUP_DIR/app_$DATE.tar.gz"
```

## Backup do banco de dados

```
docker-compose exec -T db pg_dump -U itsm_user itsm | gzip > $DB_BACKUP
```

## Backup de arquivos carregados

```
tar -czf $APP_BACKUP ./uploads
```

## Upload para S3

```
aws s3 cp $DB_BACKUP s3://itsm-backups/database/
aws s3 cp $APP_BACKUP s3://itsm-backups/files/
```

## Limpeza: manter apenas últimos 30 dias

```
find $BACKUP_DIR -type f -mtime +30 -delete
```

```
echo "Backup concluído: $DATE"
```



## 6.4 Monitoramento

# docker-compose.monitoring.yml (futuro)

```
services:
 prometheus:
 image: prom/prometheus
 ports:
 - "9090:9090"
 volumes:
 - ./prometheus.yml:/etc/prometheus/prometheus.yml

 grafana:
 image: grafana/grafana
 ports:
 - "3001:3000"
 environment:
 - GF_SECURITY_ADMIN_PASSWORD=admin

 loki:
 image: grafana/loki
 ports:
 - "3100:3100"
```

---

## ✓ 7. Plano de Implementação

### Fase 1: MVP (Mês 1-2)

#### Módulos Essenciais:

- [ ] Autenticação + LDAP (3.1, 3.2)
- [ ] Sistema de Tickets Básico (3.5)
- [ ] Inventário Simples (3.3)
- [ ] Dashboard Básico (3.9)
- [ ] Notificações por Email (3.11)

#### Entregáveis:

- Frontend em React com telas iniciais
- API FastAPI com CRUD de tickets
- Banco de dados estruturado
- Docker Compose local
- CI/CD básico

### Fase 2: Expansão (Mês 3-4)

#### Módulos Adicionados:

- [ ] Catálogo de Serviços (3.6)
- [ ] Base de Conhecimento (3.7)
- [ ] SLA e Escalonamento (3.8)

- ☐ CMDB (3.4)
- ☐ Aprovações (3.13)

### Fase 3: Maturação (Mês 5+)

#### Módulos Avançados:

- ☐ Integrações (Slack, Teams)
- ☐ Analytics avançado
- ☐ Mobile app
- ☐ AI/ML (sugestões, priorização)

---

## 8. Segurança

### Checklist de Segurança

- ☐ HTTPS obrigatório em produção
- ☐ CORS configurado restritivamente
- ☐ Rate limiting ativado
- ☐ Validação CSRF em todos os POST/PUT/DELETE
- ☐ Senhas fortes (12+ chars, complexidade)
- ☐ Sem credenciais no código (use .env)
- ☐ Logs auditados e protegidos
- ☐ Backup criptografado
- ☐ Testes de penetração periódicos

---

## 9. Documentação

### Documentos Gerados

1. **API Documentation** (Swagger/OpenAPI)
2. **User Manual** (Português)
3. **Admin Guide** (Configuração, manutenção)
4. **Developer Guide** (Setup local, arquitetura)
5. **Database Schema** (ERD)
6. **Runbooks** (Procedimentos operacionais)

---

## ☆☆ 10. Considerações Finais

Este Sistema de Gerenciamento de TI (ITSM) para o COPPEAD/UFRJ representa uma solução robusta, moderna e escalável. Com base em metodologia ITIL, arquitetura de microserviços e boas práticas de engenharia de software, está pronto para:

- ✓ **Centralizar** todos os processos de TI em uma única plataforma
- ✓ **Automatizar** tarefas repetitivas e fluxos complexos
- ✓ **Rastrear** completamente a jornada de cada ticket
- ✓ **Medir** KPIs e melhorar continuamente
- ✓ **Integrar** com sistemas existentes (GLPI, LDAP, Email)
- ✓ **Crescer** conforme necessidade da instituição

## Próximos Passos

1. Revisão executiva deste documento
2. Levantamento de requisitos específicos com stakeholders
3. Design detalhado de banco de dados (ERD)
4. Início do desenvolvimento da Fase 1
5. Testes com grupo piloto

---

**Documento versão 1.0 | 26 de janeiro de 2026**